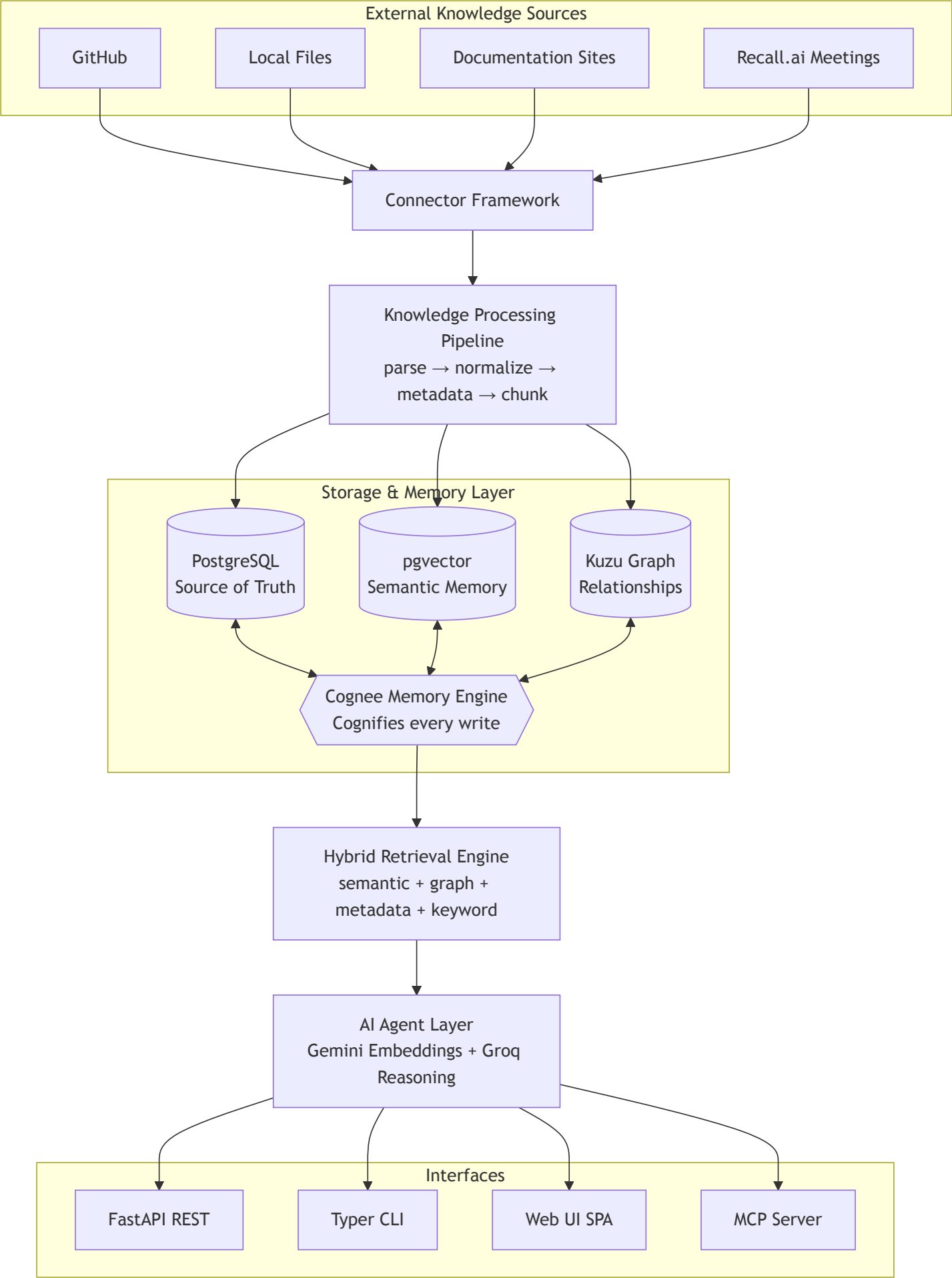


OpsMemory Architecture

OpsMemory is the operational memory layer for engineering teams: it ingests engineering knowledge, transforms it into structured operational memory, and answers questions with evidence-backed reasoning.

This document describes the implemented architecture; the [PRD](#) is the product specification, and [ADRs](#) record the decisions where implementation choices were required.

System overview



AI providers

Embeddings and reasoning are independent ports
([ADR-0006](#)): Gemini serves embeddings
(and optionally reasoning); Groq or Anthropic serve reasoning; keyless
fallbacks (hashing embeddings, extractive answers) keep every feature
functional in development. Cognee reuses the same provider keys.

Layers and packages

Layer	Package	Responsibility
API	opsmemory.api	FastAPI app, versioned routers, schemas, error envelope
CLI	opsmemory.cli	Typer commands, output rendering, API client
Services	opsmemory.services	Business logic orchestrating storage and domain
Domain	opsmemory.domain	Shared enums and value objects
Storage	opsmemory.db	SQLAlchemy models, session management, cross-dialect types
Core	opsmemory.core	Configuration, logging, error types
Connectors	opsmemory.connectors	Source integrations behind a common interface
Processing	opsmemory.processing	Deterministic knowledge pipeline stages
Retrieval	opsmemory.retrieval	Retrieval strategies, ranking, context assembly
Memory	opsmemory.memory	MemoryEngine port + Cognee adapter
Graph	opsmemory.graph	GraphStore port + Kuzu adapter
Agent	opsmemory.agent	Reasoning over curated context, citations, confidence

Storage responsibilities

- **PostgreSQL** is the *single source of truth* for all structured data: documents, entities, connectors, jobs, experiences. Everything else is

derived and rebuildable ([ADR-0003](#)).

- **pgvector** (inside the same PostgreSQL) stores embeddings for semantic retrieval. The extension is enabled by the initial migration.
- **Kuzu** stores the knowledge graph. It is embedded and single-writer, so all access goes through the `GraphStore` port and writes are confined to the worker ([ADR-0001](#)).
- **Cognee** provides long-term memory management behind the `MemoryEngine` port, configured to share PostgreSQL/pgvector rather than duplicate storage ([ADR-0002](#)).

Domain model

Core entities (all UUID-keyed, timestamped, defined in `opsmemory.db.models`):

- **Team** — owns services and repositories
- **Repository** — source of documents, deploys services
- **Service** — self-referential `depends_on` relationships
- **Document** — normalized knowledge with content hash for incremental ingestion
- **Incident** — severity/status lifecycle, root cause, resolution
- **OperationalExperience** — problem → root cause → resolution → lessons, with confidence score
- **Connector / IngestionJob** — configured sources and their sync runs

Execution principles

1. **Platform orchestrates, LLM reasons.** Request classification, routing, and retrieval strategy selection are deterministic. The LLM is invoked only for reasoning/synthesis, on curated context.
2. **Deterministic before probabilistic.** Metadata lookups and graph traversals are preferred over semantic search where they suffice.
3. **Evidence first.** Every memory and answer is traceable to its source.
4. **Incremental everything.** Content hashes and connector checkpoints make re-ingestion cheap and idempotent.

Deployment

- **Local:** `docker compose up` (Postgres + migrations + API) or `make dev`.

- **Kubernetes:** Helm chart at `deploy/helm/opsmemory` — API Deployment with liveness (`/health`) / readiness (`/ready`) probes;
`scripts/bootstrap-kind.sh` builds a complete local Kind lab.
- **MCP:** `opsmemory-mcp` (stdio) exposes ask/search/teach/graph tools to MCP clients; it is a thin client of the REST API via `OPSMemory_API_URL` .

Note: the Kuzu graph file is process-local (ADR-0001). With multiple API replicas, run graph writes in a single worker or swap the `GraphStore` adapter for a server-based graph database; the graph is rebuildable from PostgreSQL either way (ADR-0003).